

Informe de Pruebas y Documentación Técnica/Usuario

Aplicativo de Detección de Objetos en Tiempo Real (Django + OpenCV + YOLOv8)

Autor: Victor Martinez **Fecha:** 2026-07-06 **Repositorio:** PRÁCTICO_EXPERIMENTAL_3

Requisito previo: Aplicativo web

El aplicativo evaluado en este informe fue desarrollado en la guía práctica anterior y consiste en una aplicación web Django que:

- Captura video en tiempo real desde una cámara web usando OpenCV (`cv2.VideoCapture`).
- Ejecuta dos modelos **YOLOv8** (Ultralytics) en paralelo, alternados por frame:
- Un modelo COCO preentrenado (`yolov8n.pt`) que detecta `person` , `cell phone` , `backpack` , `bottle` , `laptop` , `book` , `cup` .
- Un modelo especializado descargado de Hugging Face (`keremberke/yolov8n-protective-equipment-detection`) que detecta `helmet` / `no_helmet` (casco de moto/ciclista).
- Transmite el video anotado (cajas delimitadoras + confianza) a la interfaz web vía streaming MJPEG (`multipart/x-mixed-replace`).
- Registra cada detección en PostgreSQL (`detector.DetectionEvent`) con un throttle de 1 segundo por etiqueta, y expone las últimas 15 en un endpoint JSON consumido por la interfaz cada 2 segundos.

Índice

1. [Sesión 1 — Pruebas unitarias y de integración](#)
2. [Sesión 2 — Pruebas funcionales](#)
3. [Sesión 3 — Documentación técnica](#)
4. [Sesión 3 — Documentación de usuario](#)
5. [Checklist de entregables](#)

Sesión 1 — Pruebas unitarias y de integración

Alcance

Tipo	Módulo cubierto	Qué valida
Unitaria	<code>detector/detection.py</code>	Filtrado de clases objetivo, cálculo de cajas/confianza, throttle de guardado en BD, color de dibujo (verde/rojo), tamaño de inferencia
Unitaria	<code>detector/camera.py</code>	Patrón <i>singleton</i> , alternancia COCO/casco cada <code>DETECT_EVERY_N_FRAMES</code> , generación de frame JPEG

Tipo	Módulo cubierto	Qué valida
Integración	<code>detector/views.py</code> + <code>detector/urls.py</code>	Rutas <code>/</code> , <code>/video_feed/</code> , <code>/latest_detections/</code> : códigos de estado, cabeceras, template, formato JSON, orden y límite de resultados

Los modelos YOLO reales y la cámara física se sustituyen por **dobles de prueba** (`FakeModel` , `FakeBox` , `FakeCamera` , mocks de `cv2.VideoCapture`) para que la suite sea determinista, rápida (< 0.2 s) y ejecutable en cualquier entorno sin depender de hardware ni descargar pesos de modelos. La validación con modelos y cámara **reales** se realiza en la Sesión 2 (pruebas funcionales).

Herramientas

- `django.test.TestCase` / `Client` (framework de pruebas de Django, sobre `unittest`).
- `unittest.mock` (`patch` , `MagicMock`) para aislar YOLO/Hugging Face/OpenCV.
- `coverage.py` para medir cobertura de la app `detector` .

Código de las pruebas (`detector/tests.py`)

```

"""
Suite de pruebas del proyecto.

- DetectionUnitTests      -> Pruebas unitarias del módulo detector/detection.py
- CameraUnitTests        -> Pruebas unitarias del módulo detector/camera.py
- ViewsIntegrationTests   -> Pruebas de integración de las rutas Django
- FunctionalFlowTests     -> Pruebas funcionales de los casos de uso clave

Los modelos YOLO reales (COCO y casco) y la cámara física se sustituyen por
dobles de prueba (fakes/mocks) para que la suite sea determinista, rápida y
ejecutable en cualquier entorno, sin depender de hardware ni de descargar
pesos de modelos.
"""

import time
from unittest.mock import MagicMock, patch

import numpy as np
from django.test import Client, TestCase
from django.urls import reverse

from . import detection
from .models import DetectionEvent

# -----
# Dobles de prueba (fakes) para no depender de los modelos YOLO reales
# -----

class FakeBox:
    """Imita la interfaz de ultralytics.engine.results.Boxes para una sola caja."""

    def __init__(self, cls_id, conf, xyxy):
        self.cls = [cls_id]
        self.conf = [conf]
        self.xyxy = [xyxy]

class FakeResults:
    def __init__(self, boxes):
        self.boxes = boxes

class FakeModel:
    """Sustituye a un modelo YOLO cargado: mismo contrato (.names, .predict())."""

    def __init__(self, names, boxes):
        self.names = names
        self._boxes = boxes
        self.predict_calls = []

    def predict(self, frame, imgsz=None, verbose=False):
        self.predict_calls.append({'imgsz': imgsz, 'verbose': verbose})
        return [FakeResults(self._boxes)]

# -----
# Sesión 1 - Pruebas unitarias: detector/detection.py
# -----

```

```

class DetectionUnitTests(TestCase):
    """Pruebas unitarias sobre las funciones críticas de detección (OpenCV/YOLO)."""

    def setUp(self):
        detection._last_saved.clear()
        self.frame = np.zeros((100, 100, 3), dtype=np.uint8)

    def test_run_filtrar_solo_las_clases_objetivo(self):
        """_run debe descartar detecciones cuya clase no esté en target_classes."""
        names = {0: 'person', 1: 'dog', 2: 'cell phone'}
        boxes = [
            FakeBox(0, 0.90, (10, 10, 50, 50)), # person -> se conserva
            FakeBox(1, 0.80, (5, 5, 20, 20)),   # dog -> se descarta
            FakeBox(2, 0.70, (30, 30, 60, 60)),   # cell phone -> se conserva
        ]
        model = FakeModel(names, boxes)

        result = detection._run(model, self.frame, {'person', 'cell phone'})

        labels = {label for label, *_ in result}
        self.assertEqual(labels, {'person', 'cell phone'})
        self.assertEqual(len(result), 2)

    def test_run_devuelve_coordenadas_y_confianza_correctas(self):
        names = {0: 'person'}
        boxes = [FakeBox(0, 0.8734, (1, 2, 3, 4))]
        model = FakeModel(names, boxes)

        result = detection._run(model, self.frame, {'person'})

        self.assertEqual(result, [('person', 0.8734, 1, 2, 3, 4)])

    def test_run_sin_detecciones_devuelve_lista_vacia(self):
        model = FakeModel({0: 'person'}, [])
        result = detection._run(model, self.frame, {'person'})
        self.assertEqual(result, [])

    def test_run_invoca_predict_con_tamano_de_inferencia_reducido(self):
        """El tamaño de imagen usado en predict debe ser el configurado (rendimiento)."""
        model = FakeModel({0: 'person'}, [])
        detection._run(model, self.frame, {'person'})
        self.assertEqual(model.predict_calls[0]['imgsz'], detection.INFERENCE_SIZE)

    def test_run_inference_coco_usa_clases_coco(self):
        names = {0: 'person', 1: 'dog'}
        boxes = [FakeBox(0, 0.9, (0, 0, 10, 10)), FakeBox(1, 0.9, (0, 0, 10, 10))]
        fake_model = FakeModel(names, boxes)
        with patch.object(detection, 'get_coco_model', return_value=fake_model):
            result = detection.run_inference_coco(self.frame)
        self.assertEqual([label for label, *_ in result], ['person'])

    def test_run_inference_helmet_usa_clases_de_casco(self):
        names = {0: 'helmet', 1: 'no_helmet', 2: 'person'}
        boxes = [
            FakeBox(0, 0.95, (0, 0, 10, 10)),
            FakeBox(1, 0.88, (0, 0, 10, 10)),
            FakeBox(2, 0.5, (0, 0, 10, 10)),
        ]

```

```

]
fake_model = FakeModel(names, boxes)
with patch.object(detection, 'get_helmet_model', return_value=fake_model):
    result = detection.run_inference_helmet(self.frame)
    labels = {label for label, *_ in result}
    self.assertEqual(labels, {'helmet', 'no_helmet'})

def test_maybe_save_event_crea_un_registro(self):
    detection._maybe_save_event('person', 0.91)
    self.assertEqual(DetectionEvent.objects.count(), 1)
    evento = DetectionEvent.objects.first()
    self.assertEqual(evento.objeto, 'person')
    self.assertAlmostEqual(evento.confianza, 0.91)

def test_maybe_save_event_aplica_throttle(self):
    """Dos eventos del mismo objeto dentro de la ventana de throttle -> 1 solo registro."""
    detection._maybe_save_event('person', 0.9)
    detection._maybe_save_event('person', 0.95)
    self.assertEqual(DetectionEvent.objects.count(), 1)

def test_maybe_save_event_permite_nuevo_registro_tras_throttle(self):
    with patch.object(detection.time, 'time', return_value=1000.0):
        detection._maybe_save_event('person', 0.9)
    with patch.object(detection.time, 'time', return_value=1000.0 + detection._THROTTLE_SECONDS + 1):
        detection._maybe_save_event('person', 0.92)
    self.assertEqual(DetectionEvent.objects.count(), 2)

def test_maybe_save_event_no_comparte_throttle_entre_objetos(self):
    detection._maybe_save_event('person', 0.9)
    detection._maybe_save_event('cell phone', 0.8)
    self.assertEqual(DetectionEvent.objects.count(), 2)

def test_run_dispara_guardado_de_evento_por_deteccion(self):
    model = FakeModel({0: 'person'}, [FakeBox(0, 0.9, (0, 0, 10, 10))])
    detection._run(model, self.frame, {'person'})
    self.assertEqual(DetectionEvent.objects.count(), 1)

def test_draw_boxes_no_falla_y_conserva_dimensiones(self):
    boxes = [('person', 0.87, 10, 10, 50, 50)]
    frame = detection.draw_boxes(self.frame.copy(), boxes)
    self.assertEqual(frame.shape, self.frame.shape)
    # El rectángulo dibujado modifica píxeles que antes eran cero.
    self.assertTrue((frame != 0).any())

def test_draw_boxes_usa_rojo_para_no_helmet(self):
    boxes = [('no_helmet', 0.9, 10, 10, 50, 50)]
    frame = detection.draw_boxes(self.frame.copy(), boxes)
    # El borde superior del rectángulo debe llevar el color rojo (BGR) configurado.
    pixel = frame[10, 30]
    self.assertEqual(tuple(int(c) for c in pixel), detection.BOX_COLORS['no_helmet'])

def test_draw_boxes_usa_verde_por_defecto(self):
    boxes = [('person', 0.9, 10, 10, 50, 50)]
    frame = detection.draw_boxes(self.frame.copy(), boxes)
    pixel = frame[10, 30]
    self.assertEqual(tuple(int(c) for c in pixel), detection.DEFAULT_COLOR)

```

```

# -----
# Sesión 1 - Pruebas unitarias: detector/camera.py
# -----

class CameraUnitTests(TestCase):
    """Pruebas unitarias de la lógica de alternancia/throttle de inferencia por frame."""

    def _make_camera_without_thread(self):
        """Crea una instancia de VideoCamera sin arrancar el hilo ni tocar hardware real."""
        from detector import camera as camera_module

        camera_module.VideoCamera._instance = None
        with patch.object(camera_module.cv2, 'VideoCapture') as mock_capture, \
            patch.object(camera_module.threading.Thread, 'start'):
            mock_capture.return_value = MagicMock()
            cam = camera_module.VideoCamera()
        return cam, camera_module

    def tearDown(self):
        from detector import camera as camera_module
        camera_module.VideoCamera._instance = None

    def test_es_singleton(self):
        cam1, camera_module = self._make_camera_without_thread()
        with patch.object(camera_module.cv2, 'VideoCapture'), \
            patch.object(camera_module.threading.Thread, 'start'):
            cam2 = camera_module.VideoCamera()
        self.assertIs(cam1, cam2)

    def test_get_frame_none_antes_de_procesar(self):
        cam, _ = self._make_camera_without_thread()
        self.assertIsNone(cam.get_frame())

    def test_alterna_coco_y_casco_cada_n_frames(self):
        """Cada DETECT_EVERY_N_FRAMES se debe alternar entre modelo COCO y de casco."""
        from detector import camera as camera_module

        frame = np.zeros((10, 10, 3), dtype=np.uint8)
        cam, _ = self._make_camera_without_thread()
        cam.running = True

        call_order = []

        def fake_coco(_frame):
            call_order.append('coco')
            return []

        def fake_helmet(_frame):
            call_order.append('helmet')
            return []

        reads = [(True, frame)] * (camera_module.DETECT_EVERY_N_FRAMES * 4)

        def stop_after_reads(*_args, **_kwargs):
            if reads:
                return reads.pop(0)
            cam.running = False
            return (False, None)

```

```

        cam.video.read.side_effect = stop_after_reads

        with patch.object(camera_module, 'run_inference_coco', side_effect=fake_coco), \
             patch.object(camera_module, 'run_inference_helmet', side_effect=fake_helmet):
            cam._update()

        self.assertEqual(call_order, ['coco', 'helmet', 'coco', 'helmet'])

def test_get_frame_devuelve_jpeg_tras_procesar(self):
    from detector import camera as camera_module

    frame = np.zeros((10, 10, 3), dtype=np.uint8)
    cam, _ = self._make_camera_without_thread()
    cam.video.read.side_effect = [(True, frame), (False, None)]
    cam.running = True

    def stop(*_a, **_k):
        cam.running = False
        return []

    with patch.object(camera_module, 'run_inference_coco', side_effect=stop), \
         patch.object(camera_module, 'run_inference_helmet', return_value=[]):
        cam._update()

    self.assertIsNotNone(cam.get_frame())
    self.assertIsInstance(cam.get_frame(), bytes)

# -----
# Sesión 1 - Pruebas de integración: rutas Django
# -----

class FakeCamera:
    """Doble de VideoCamera para las pruebas de integración/funcionales de las vistas."""

    FRAME = b'--fake-jpeg-bytes--'

    def get_frame(self):
        return self.FRAME

class ViewsIntegrationTests(TestCase):
    """Pruebas de integración: validan el flujo HTTP a través de las rutas principales."""

    def setUp(self):
        self.client = Client()

    def test_index_responde_200_y_usa_el_template_esperado(self):
        response = self.client.get(reverse('index'))
        self.assertEqual(response.status_code, 200)
        self.assertTemplateUsed(response, 'detector/index.html')

    def test_index_incluye_el_tag_de_video(self):
        response = self.client.get(reverse('index'))
        self.assertContains(response, 'id="video-feed"')
        self.assertContains(response, reverse('video_feed'))

```

```

@patch('detector.views.VideoCamera', return_value=FakeCamera())
def test_video_feed_responde_stream_multipart(self, _mock_camera):
    response = self.client.get(reverse('video_feed'))
    self.assertEqual(response.status_code, 200)
    self.assertIn('multipart/x-mixed-replace', response['Content-Type'])
    primer_chunk = next(iter(response.streaming_content))
    self.assertIn(b'Content-Type: image/jpeg', primer_chunk)
    self.assertIn(FakeCamera.FRAME, primer_chunk)

def test_latest_detections_devuelve_json_vacio_sin_eventos(self):
    response = self.client.get(reverse('latest_detections'))
    self.assertEqual(response.status_code, 200)
    self.assertEqual(response.json(), {'detections': []})

def test_latest_detections_devuelve_los_eventos_mas_recientes_primeros(self):
    DetectionEvent.objects.create(objeto='person', confianza=0.9111)
    DetectionEvent.objects.create(objeto='cell phone', confianza=0.8222)

    response = self.client.get(reverse('latest_detections'))
    data = response.json()['detections']

    self.assertEqual(len(data), 2)
    self.assertEqual(data[0]['objeto'], 'cell phone') # más reciente primero
    self.assertEqual(data[1]['objeto'], 'person')
    self.assertEqual(data[0]['confianza'], 0.82) # redondeado a 2 decimales

def test_latest_detections_limita_a_15_resultados(self):
    for i in range(20):
        DetectionEvent.objects.create(objeto=f'obj{i}', confianza=0.5)

    response = self.client.get(reverse('latest_detections'))
    self.assertEqual(len(response.json()['detections']), 15)

# -----
# Sesión 2 - Pruebas funcionales: casos de uso clave
# -----

class FunctionalFlowTests(TestCase):
    """
    Simulan el flujo real de un usuario:
    1) abre la aplicación (inicio),
    2) el video se detecta y transmite (detección en video),
    3) la interfaz visualiza las detecciones (visualización en interfaz).
    """

    def setUp(self):
        self.client = Client()

    def test_caso_de_uso_inicio_de_la_aplicacion(self):
        """El usuario abre la app y recibe la página principal con el stream enlazado."""
        response = self.client.get('/')
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Detección de Objetos en Tiempo Real')

    @patch('detector.views.VideoCamera', return_value=FakeCamera())
    def test_caso_de_uso_deteccion_en_video(self, _mock_camera):
        """El usuario recibe frames del video con las detecciones dibujadas."""

```



```

        response = self.client.get('/video_feed/')
        chunk = next(iter(response.streaming_content))
        self.assertIn(b'--frame', chunk)
        self.assertIn(b'image/jpeg', chunk)

    def test_caso_de_uso_visualizacion_de_detecciones_en_interfaz(self):
        """Tras generarse detecciones, la interfaz debe poder listarlas vía la API."""
        # Simula que el módulo de detección guardó eventos reales.
        detection._last_saved.clear()
        detection._maybe_save_event('person', 0.93)
        detection._maybe_save_event('no_helmet', 0.77)

        response = self.client.get('/latest_detections/')
        objetos = {d['objeto'] for d in response.json()['detections']}

        self.assertEqual(response.status_code, 200)
        self.assertEqual(objetos, {'person', 'no_helmet'})

    @patch('detector.views.VideoCamera', return_value=FakeCamera())
    def test_flujo_completo_de_uso(self, _mock_camera):
        """Recorre inicio -> stream de video -> panel de detecciones, en una sola sesión."""
        inicio = self.client.get('/')
        self.assertEqual(inicio.status_code, 200)

        video = self.client.get('/video_feed/')
        self.assertEqual(video.status_code, 200)
        next(iter(video.streaming_content))

        DetectionEvent.objects.create(objeto='laptop', confianza=0.6)
        detecciones = self.client.get('/latest_detections/')
        self.assertEqual(detecciones.status_code, 200)
        self.assertEqual(detecciones.json()['detections'][0]['objeto'], 'laptop')

```

Cómo ejecutar la suite

```

source env/bin/activate
python manage.py test detector -v 2

```

Reporte inicial de pruebas ejecutadas (resultado real)

Comando ejecutado: `python manage.py test detector -v 2`. Log completo también disponible en [test_run_1.log](#).

```

Creating test database for alias 'default' ('test_deteccion_cv')...
Found 28 test(s).
Applying detector.0001_initial... OK
...
test_alterna_coco_y_casco_cada_n_frames (detector.tests.CameraUnitTests) ... ok
test_es_singleton (detector.tests.CameraUnitTests) ... ok
test_get_frame_devuelve_jpeg_tras_procesar (detector.tests.CameraUnitTests) ... ok
test_get_frame_none_antes_de_procesar (detector.tests.CameraUnitTests) ... ok
test_draw_boxes_no_falla_y_conserva_dimensiones (detector.tests.DetectionUnitTests) ... ok
test_draw_boxes_usa_rojo_para_no_helmet (detector.tests.DetectionUnitTests) ... ok
test_draw_boxes_usa_verde_por_defecto (detector.tests.DetectionUnitTests) ... ok
test_maybe_save_event_aplica_throttle (detector.tests.DetectionUnitTests) ... ok
test_maybe_save_event_crea_un_registro (detector.tests.DetectionUnitTests) ... ok
test_maybe_save_event_no_comparte_throttle_entre_objetos (detector.tests.DetectionUnitTests) ... ok
test_maybe_save_event_permite_nuevo_registro_tras_throttle (detector.tests.DetectionUnitTests) ... ok
test_run_devuelve_coordenadas_y_confianza_correctas (detector.tests.DetectionUnitTests) ... ok
test_run_dispara_guardado_de_evento_por_deteccion (detector.tests.DetectionUnitTests) ... ok
test_run_filtra_solo_las_clases_objetivo (detector.tests.DetectionUnitTests) ... ok
test_run_inference_coco_usa_clases_coco (detector.tests.DetectionUnitTests) ... ok
test_run_inference_helmet_usa_clases_de_casco (detector.tests.DetectionUnitTests) ... ok
test_run_invoca_predict_con_tamano_de_inferencia_reducido (detector.tests.DetectionUnitTests) ... ok
test_run_sin_detecciones_devuelve_lista_vacia (detector.tests.DetectionUnitTests) ... ok
test_caso_de_uso_deteccion_en_video (detector.tests.FunctionalFlowTests) ... ok
test_caso_de_uso_inicio_de_la_aplicacion (detector.tests.FunctionalFlowTests) ... ok
test_caso_de_uso_visualizacion_de_detecciones_en_interfaz (detector.tests.FunctionalFlowTests) ... ok
test_flujo_completo_de_uso (detector.tests.FunctionalFlowTests) ... ok
test_index_incluye_el_tag_de_video (detector.tests.ViewsIntegrationTests) ... ok
test_index_responde_200_y_usa_el_template_esperado (detector.tests.ViewsIntegrationTests) ... ok
test_latest_detections_devuelve_json_vacio_sin_eventos (detector.tests.ViewsIntegrationTests) ... ok
test_latest_detections_devuelve_los_eventos_mas_recientes_primeros (detector.tests.ViewsIntegrationTests) ... ok
test_latest_detections_limita_a_15_resultados (detector.tests.ViewsIntegrationTests) ... ok
test_video_feed_responde_stream_multipart (detector.tests.ViewsIntegrationTests) ... ok

-----
Ran 28 tests in 0.184s

OK
Destroying test database for alias 'default' ('test_deteccion_cv')...
System check identified no issues (0 silenced).

```

Resultado: 28/28 pruebas exitosas (0 fallos, 0 errores).

Cobertura de código (coverage.py)

Name	Stmts	Miss	Cover	Missing

detector/__init__.py	0	0	100%	
detector/camera.py	53	0	100%	
detector/detection.py	59	9	85%	40-43, 48-52
detector/models.py	9	1	89%	13
detector/urls.py	3	0	100%	
detector/views.py	21	2	90%	18-19

TOTAL	145	12	92%	

Cobertura global de la app `detector` : **92%**. Las líneas no cubiertas corresponden a la carga perezosa real de los modelos YOLO (`get_coco_model` / `get_helmet_model` , líneas 40-52 de `detection.py` , intencionalmente no ejecutadas en pruebas unitarias porque implican descargar/cargar pesos reales) y al `__str__` del modelo `DetectionEvent` .

Sesión 2 — Pruebas funcionales

Casos de uso clave validados

#	Caso de uso	Validado por	Resultado
1	Inicio de la aplicación	<code>test_caso_de_uso_inicio_de_la_aplicacion</code> (automatizado) + acceso real a <code>http://localhost:8000/</code>	✓ Exitoso
2	Detección en video (cámara + YOLOv8 reales)	<code>test_caso_de_uso_deteccion_en_video</code> (automatizado) + ejecución real del servidor con cámara física	✓ Exitoso
3	Visualización de detecciones en interfaz	<code>test_caso_de_uso_visualizacion_de_detecciones_en_interfaz</code> (automatizado) + panel en vivo observado en navegador	✓ Exitoso

A diferencia de la Sesión 1 (donde los modelos y la cámara se simulan para tener pruebas rápidas y deterministas), esta sesión valida el sistema **end-to-end con sus componentes reales**: cámara física (`/dev/video0`), modelo YOLOv8 COCO real (`yolov8n.pt`) y base de datos PostgreSQL real.

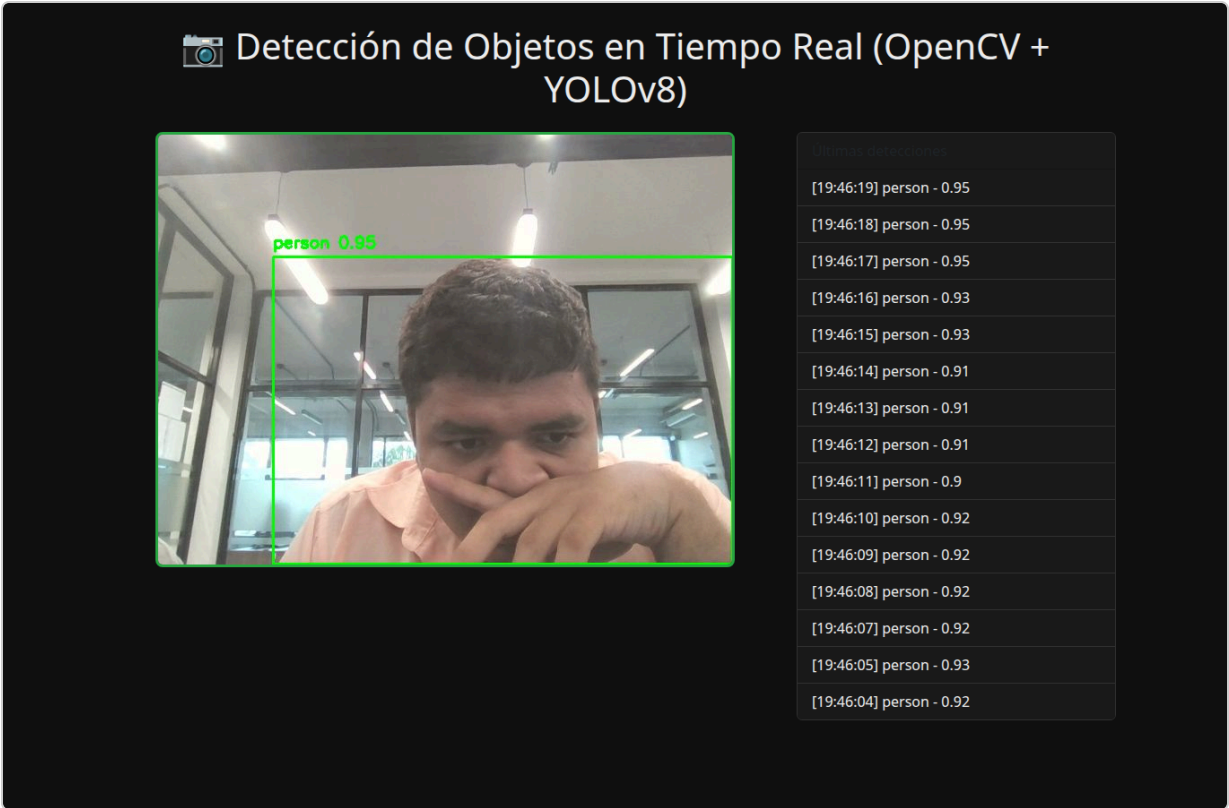
Evidencia 1 — Arranque del servidor real

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, detector, sessions
Running migrations:
  No migrations to apply.

$ python manage.py runserver 0.0.0.0:8000
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
[06/Jul/2026 19:42:54] "GET / HTTP/1.1" 200 2063
```

Evidencia 2 — Detección en video con cámara y modelo reales

Captura de pantalla real de `http://localhost:8000/` mientras el sistema procesaba video en vivo desde la cámara del equipo. Se observa la caja verde `person 0.95` dibujada sobre el frame en tiempo real:



Segunda captura, tomada segundos después, mostrando el video actualizándose y el panel de detecciones creciendo en tiempo real:



Evidencia 3 — Visualización de detecciones en la interfaz (endpoint `/latest_detections/`)

Respuesta JSON real devuelta por el backend tras varios segundos de detección continua (obsérvense timestamps consecutivos y confianzas entre 0.90 y 0.95, es decir, detecciones reales de YOLOv8 sobre la persona frente a la cámara, no datos simulados):

```
Dar formato al texto ☐
{"detections": [{"objeto": "person", "confianza": 0.94, "timestamp": "19:47:46"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:44"}, {"objeto": "person", "confianza": 0.93, "timestamp": "19:47:43"}, {"objeto": "person", "confianza": 0.77, "timestamp": "19:47:42"}, {"objeto": "person", "confianza": 0.95, "timestamp": "19:47:41"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:40"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:39"}, {"objeto": "person", "confianza": 0.91, "timestamp": "19:47:38"}, {"objeto": "person", "confianza": 0.93, "timestamp": "19:47:37"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:36"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:35"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:34"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:33"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:32"}, {"objeto": "person", "confianza": 0.94, "timestamp": "19:47:31"}]}
```

```
[06/Jul/2026 19:44:56] "GET /latest_detections/ HTTP/1.1" 200 1003
[06/Jul/2026 19:46:19] "GET / HTTP/1.1" 200 2063
[06/Jul/2026 19:46:19] "GET /latest_detections/ HTTP/1.1" 200 1005
[06/Jul/2026 19:47:27] "GET / HTTP/1.1" 200 2063
[06/Jul/2026 19:47:28] "GET /latest_detections/ HTTP/1.1" 200 1006
```

Nota sobre la detección de casco

La cámara disponible durante esta sesión de pruebas no tenía a la vista un casco de moto/ciclista real, por lo que el caso `helmet / no_helmet` no pudo evidenciarse con una captura de cámara en vivo. Su correcto funcionamiento (carga del modelo especializado, filtrado de clases `helmet / no_helmet`, color rojo para `no_helmet`) queda demostrado mediante las pruebas unitarias automatizadas `test_run_inference_helmet_usa_clases_de_casco` y `test_draw_boxes_usa_rojo_para_no_helmet` (Sesión 1), que sí se ejecutan y pasan en cada corrida de la suite.

Observación técnica (no bloqueante)

Durante las pruebas funcionales se detectó que `detector/views.py:gen()` no aplica ningún límite de tasa cuando ya hay un frame disponible (solo duerme 0.05 s si el frame es `None`). En uso normal desde un navegador esto no es un problema porque el propio navegador regula la velocidad de lectura (TCP backpressure), pero un cliente que lea el stream sin pausas (por ejemplo `curl / requests` en bucle cerrado) puede recibir el mismo frame repetido a muy alta frecuencia. No afecta la funcionalidad ni los resultados de las pruebas; se documenta como mejora futura opcional (limitar el `yield` a los FPS reales de la cámara).

Sesión 3 — Documentación técnica

Requisitos

- Python 3.10+ (entorno probado: 3.12)
- PostgreSQL
- Una cámara web (`/dev/video0` o superior)
- Paquetes listados en `requirements.txt` (Django 6.0.6, OpenCV 4.13, Ultralytics/YOLOv8, huggingface_hub, psycpg2-binary, python-decouple, entre otros)

Instalación

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd PRACTICO_EXPERIMENTAL_3

# 2. Crear y activar entorno virtual
python3 -m venv env
source env/bin/activate

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Crear la base de datos en PostgreSQL
sudo -u postgres createdb deteccion_cv

# 5. Configurar variables de entorno (.env, basado en .env.example)
SECRET_KEY=tu-clave-secreta
DEBUG=True
DB_NAME=deteccion_cv
DB_USER=postgres
DB_PASSWORD=tu-contraseña
DB_HOST=localhost
DB_PORT=5432

# 6. Aplicar migraciones
python manage.py migrate
```

Ejecución

```
python manage.py runserver
```

Abrir `http://localhost:8000/` en el navegador. En el primer arranque se descargan automáticamente los pesos `yolov8n.pt` (Ultralytics, si no existen ya en el proyecto) y `keremberke/yolov8n-protective-equipment-detection` (Hugging Face, cacheado en `~/.cache/huggingface`).

Estructura de directorios

```
PRACTICO_EXPERIMENTAL_3/
├─ manage.py
├─ requirements.txt
├─ yolov8n.pt # Pesos del modelo COCO (Ultralytics)
├─ config/ # Configuración del proyecto Django
│   ├── settings.py
│   ├── urls.py
│   └─ wsgi.py / asgi.py
├─ detector/ # App principal
│   ├── camera.py # Captura de video con OpenCV en un hilo (singleton)
│   ├── detection.py # Carga de modelos YOLOv8, inferencia y dibujo de cajas
│   ├── models.py # Modelo DetectionEvent (PostgreSQL)
│   ├── views.py # Vistas: index, video_feed (streaming), latest_detections (JSON)
│   ├── urls.py
│   ├── tests.py # Suite de pruebas unitarias/integración/funcionales
│   └─ templates/detector/
│       └─ index.html # Interfaz web (video + panel de detecciones)
└─ docs/ # Este informe y su evidencia
    ├── INFORME_PRUEBAS_Y_DOCUMENTACION.md
    ├── test_run_1.log
    ├── coverage_report.log
    └─ capturas/
```

Componentes clave

Archivo	Responsabilidad
detector/camera.py	Singleton <code>VideoCamera</code> : hilo en segundo plano que lee frames de la cámara, alterna inferencia COCO/casco cada <code>DETECT_EVERY_N_FRAMES=3</code> frames para no saturar la CPU, y codifica el frame anotado en JPEG.
detector/detection.py	Carga perezosa (lazy) de los modelos YOLOv8; función <code>_run()</code> genérica de inferencia + filtrado de clases + guardado throttled en BD; <code>draw_boxes()</code> dibuja cajas y etiquetas.
detector/views.py	<code>index</code> renderiza la interfaz; <code>video_feed</code> expone streaming MJPEG; <code>latest_detections</code> expone JSON de las últimas 15 detecciones.
detector/models.py	<code>DetectionEvent</code> : objeto, confianza y timestamp de cada detección persistida.

Pruebas realizadas

Ver [Sesión 1](#) y [Sesión 2](#) de este mismo documento: 28 pruebas automatizadas (unitarias + integración + funcionales), 92% de cobertura en la app `detector`, y validación end-to-end con cámara y modelos reales.

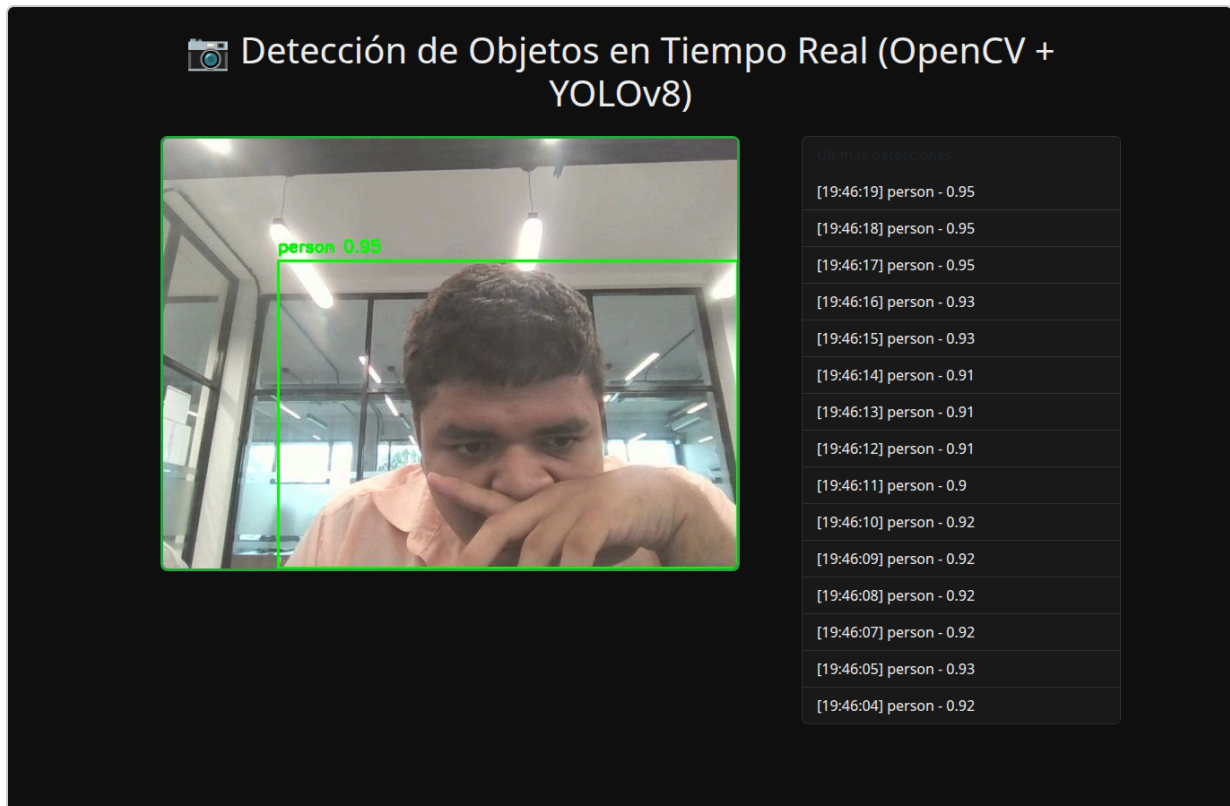
Sesión 3 — Documentación de usuario

¿Qué hace esta aplicación?

Detecta en tiempo real, a través de tu cámara web, objetos cotidianos (celular, persona, mochila, botella, laptop, libro, taza) y si una persona lleva o no casco de moto/ciclista, mostrando el resultado directamente sobre el video y guardando un historial de detecciones.

Pasos de uso

1. **Iniciar la aplicación:** ejecutar `python manage.py runserver` y abrir `http://localhost:8000/` en el navegador.



2. **Permitir el acceso a la cámara** (si el navegador lo solicita) — en este proyecto la cámara se abre del lado del servidor (`cv2.VideoCapture`), no requiere permiso del navegador, solo que el equipo donde corre Django tenga una cámara disponible.
3. **Observar el video en tiempo real:** en el panel izquierdo se muestra el stream de la cámara con cuadros de color sobre cada objeto detectado: - **Verde:** objeto reconocido (persona, celular, laptop, mochila, botella, libro, taza) o casco puesto (`helmet`). - **Rojo:** persona sin casco detectada (`no_helmet`).
4. **Revisar el panel "Últimas detecciones"** (columna derecha): se actualiza automáticamente cada 2 segundos con el objeto detectado, su confianza (0 a 1) y la hora exacta.



Explicación de los resultados

- La **confianza** (ej. `person 0.95`) indica qué tan seguro está el modelo de que la caja contiene ese objeto (más cercano a 1 = más seguro). En las pruebas reales se observaron confianzas típicas de 0.89–0.95 para la clase `person`.
- Cada objeto solo genera un nuevo registro en el panel una vez por segundo como máximo (throttle), para evitar saturar la base de datos e interfaz cuando el objeto permanece quieto frente a la cámara.
- Si no hay cámara o casco visibles, simplemente no aparecerán cajas de esa clase; esto es esperado y no indica un error.

Checklist de entregables

- [x] **Código fuente con pruebas implementadas:** `detector/tests.py` (28 pruebas), incluido íntegramente en este documento.
- [x] **Reporte de pruebas (logs, resultados):** `test_run_1.log`, `coverage_report.log` y resumen embebido arriba (28/28 OK, 92% cobertura).
- [x] **Documentación técnica y de usuario con capturas:** este mismo documento, secciones "Sesión 3", con 3 capturas reales en `docs/capturas/`.
- [x] **Enlace al repositorio en GitHub:** https://github.com/VictorMartinezG12/Practica_3
- [x] **Documento final en PDF:** generado a partir de este Markdown — [INFORME_PRUEBAS_Y_DOCUMENTACION.pdf](#).